AI Assistant Coding

Lab - 6.3

Question 1:

You are developing a simple student information management module.
Task
• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
• The class should include attributes such as name, roll number, and branch.
• Add a method display_details() to print student information.
• Execute the code and verify the output.
• Analyze the code generated by the AI tool for correctness and clarity.
Expected Output #1
• A Python class with a constructor (__init__) and a display_details() method.
• Sample object creation and output displayed on the console.
• Brief analysis of AI-generated code.

Prompt :

```
#Generate a Python Student class with attributes name, roll_number, and branch.Include a constructor (__init__) and a method display_details() that prints student information. Create a sample object and display its details.
```

Code :

```
#Generate a Python Student class with attributes name, roll_number, and branch.Include a constructor (__init__) and a method display_details() that prints student information. Create a sample object and display its details.

class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")
```

```python
# Example usage
name = input("Enter student name: ")
roll_number = input("Enter roll number: ")
branch = input("Enter branch: ")
student = Student(name, roll_number, branch)
student.display_details()
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab5.py"
Enter student name: siri
Enter roll number: 1236
Enter branch: cse
Student Name: siri
Roll Number: 1236
Branch: cse
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis :

The Student class correctly initializes attributes using the constructor and displays details using a dedicated method.The code is clear, readable, and follows proper object-oriented design without logical errors.

Question 2:

You are writing a utility function to display multiples of a given number.
Task
• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
• Analyze the generated loop logic.
• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,
while instead of for).
Expected Output #2
• Correct loop-based Python implementation.
• Output showing the first 10 multiples of a number.
• Comparison and analysis of different looping approaches.

Prompt :

```python
#Write a Python function that prints the first 10 multiples of a given number using a for loop.Take the number as input and display the output.
```

Code :

```
#Write a Python function that prints the first 10 multiples of a given number using a for loop.Take the number as input and display the output.
def print_multiples(number):
    for i in range(1, 11):
        print(f"{number} x {i} = {number * i}")
# Example usage
num = int(input("Enter a number to print its first 10 multiples: "))
print_multiples(num)
```

Output :

```
Enter a number to print its first 10 multiples: 78
78 x 1 = 78
78 x 2 = 156
78 x 3 = 234
78 x 4 = 312
78 x 5 = 390
78 x 6 = 468
78 x 7 = 546
78 x 8 = 624
78 x 9 = 702
78 x 10 = 780
PS C:\Users\dasar\AI-Assistant-Coding>
```

Prompt :

```
#Rewrite the function to print the first 10 multiples of a given number using a while loop.
```

Code :

```
#Rewrite the function to print the first 10 multiples of a given number using a while loop.
def print_multiples_while(number):
    i = 1
    while i <= 10:
        print(f"{number} x {i} = {number * i}")
        i += 1
# Example usage
num = int(input("Enter a number to print its first 10 multiples using while loop: "))
print_multiples_while(num)
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab5.py"
Enter a number to print its first 10 multiples using while loop: 76
76 x 1 = 76
76 x 2 = 152
76 x 3 = 228
76 x 4 = 304
76 x 5 = 380
76 x 6 = 456
76 x 7 = 532
76 x 8 = 608
76 x 9 = 684
76 x 10 = 760
```

Analysis :

Both `for` and `while` loop implementations correctly generate the first 10 multiples of a given number. The logic is simple, efficient, and demonstrates correct use of iterative control structures.

Question 3:

You are building a basic classification system based on age.
Task
• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
• Analyze the generated conditions and logic.
• Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).
Expected Output #3
• A Python function that classifies age into appropriate groups.
• Clear and correct conditional logic.
• Explanation of how the conditions work.

Prompt :

```
#Write a Python function that classifies a person based on age using
if-elif-else conditions: child, teenager, adult, and senior.
```

Code :

```
#Write a Python function that classifies a person based on age using
if-elif-else conditions: child, teenager, adult, and senior.
def classify_age(age):
    if age < 13:
        return "Child"
    elif 13 <= age < 20:
        return "Teenager"
```

```
    elif 20 <= age < 60:
        return "Adult"
    else:
        return "Senior"
# Example usage
age = int(input("Enter your age: "))
category = classify_age(age)
print(f"You are classified as: {category}")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab5.py"
Enter your age: 21
You are classified as: Adult
```

Prompt :

```
#Implement the same age classification using a simplified or
alternative conditional approach (such as range checks or
dictionary-based logic).
```

Code :

```
#Implement the same age classification using a simplified or
alternative conditional approach (such as range checks or
dictionary-based logic).
def classify_age_alternative(age):
    age_ranges = {
        range(0, 13): "Child",
        range(13, 20): "Teenager",
        range(20, 60): "Adult",
        range(60, 150): "Senior"
    }
    for age_range, category in age_ranges.items():
        if age in age_range:
            return category
    return "Invalid age"
# Example usage
age = int(input("Enter your age for alternative classification: "))
category = classify_age_alternative(age)
print(f"You are classified as: {category}")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab5.py"
Enter your age for alternative classification: 21
You are classified as: Adult
```

Analysis :

The `if-elif-else` approach correctly classifies age based on defined ranges with clear conditions. The dictionary-based alternative improves readability and avoids repetitive conditional checks.

Question 4:

You need to calculate the sum of the first n natural numbers.
Task
• Use AI assistance to generate a sum_to_n() function using a for loop.
• Analyze the generated code.
• Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.
Expected Output #4
• Python function to compute the sum of first n numbers.
• Correct output for sample inputs.
• Explanation and comparison of different approaches.

Prompt :

```
#Generate a Python function sum_to_n(n) that calculates the sum of the first n natural numbers using a for loop.
```

Code :

```
#Generate a Python function sum_to_n(n) that calculates the sum of the first n natural numbers using a for loop.
def sum_to_n(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
# Example usage
n = int(input("Enter a number n to calculate the sum of first n natural numbers: "))
result = sum_to_n(n)
print(f"The sum of the first {n} natural numbers is: {result}"
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab5.py"
Enter a number n to calculate the sum of first n natural numbers: 50
The sum of the first 50 natural numbers is: 1275
```

Prompt :

```
#Provide an alternative implementation of sum_to_n(n) using a while
loop or a mathematical formula.
```

Code :

```
#Provide an alternative implementation of sum_to_n(n) using a while
loop or a mathematical formula.
def sum_to_n_alternative(n):
    # Using the mathematical formula n * (n + 1) / 2
    return n * (n + 1) // 2
# Example usage
n = int(input("Enter a number n to calculate the sum of first n natural
numbers using alternative method: "))
result = sum_to_n_alternative(n)
print(f"The sum of the first {n} natural numbers is: {result}")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab5.py"
Enter a number n to calculate the sum of first n natural numbers using alternative method: 50
The sum of the first 50 natural numbers is: 1275
```

Analysis :

The loop-based solution correctly accumulates the sum using iteration and produces accurate results. The formula-based approach is more efficient with constant time complexity and simpler logic.

Question 5:

Scenario
You are designing a basic banking application.
Task
• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
• Analyze the AI-generated class structure and logic.

• Add meaningful comments and explain the working of the code.
Expected Output #5
• Complete Python Bank Account class.
• Demonstration of deposit and withdrawal operations with updated balance.
• Well-commented code with a clear explanation.

Prompt :

```
#Generate a Python program for a simple bank account system using
class, loops, and conditional statements
```

Code :

```
#Generate a Python program for a simple bank account system using
class, loops, and conditional statements
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount}. New balance:
${self.balance}")
        else:
            print("Deposit amount must be positive.")
    def withdraw(self, amount):
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew: ${amount}. New balance:
${self.balance}")
            else:
                print("Insufficient funds.")
        else:
            print("Withdrawal amount must be positive.")
    def get_balance(self):
        return self.balance
# Example usage
account_holder = input("Enter account holder name: ")
account = BankAccount(account_holder)
while True:
    print("\nOptions: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit")
```

```
    choice = input("Choose an option (1-4): ")
    if choice == '1':
        amount = float(input("Enter amount to deposit: "))
        account.deposit(amount)
    elif choice == '2':
        amount = float(input("Enter amount to withdraw: "))
        account.withdraw(amount)
    elif choice == '3':
        balance = account.get_balance()
        print(f"Current balance: ${balance}")
    elif choice == '4':
        print("Exiting the program.")
        break
    else:
        print("Invalid option. Please try again.")
```

Output :

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab4.py"
Enter account holder name: siri

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 2
Enter amount to withdraw: 1500
Insufficient funds.

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 1
Enter amount to deposit: 2000
Deposited: $2000.0. New balance: $2000.0

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 2
Enter amount to withdraw: 1500
Withdrew: $1500.0. New balance: $500.0

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 3
Current balance: $500.0

Options: 1. Deposit 2. Withdraw 3. Check Balance 4. Exit
Choose an option (1-4): 4
Exiting the program.
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis :

The bank account program correctly integrates classes, loops, and conditionals for user interaction. It functions as expected but does not handle invalid numeric input or data persistence.